
Bases de Datos

Documental
MongoDB

¿Qué es MongoDB?

MongoDB es un código abierto, es un sistema de gestión de bases de datos no relacionales (SGBD) que utiliza documentos flexibles en lugar de tablas y filas para procesar y almacenar diversas formas de datos.

Como solución de base de datos NoSQL, MongoDB no requiere un sistema de gestión de bases de datos relacionales (SGBDR), por lo que proporciona un modelo de almacenamiento de datos elástico que permite a los usuarios almacenar y consultar tipos de datos multivariados con facilidad. Esto no solo simplifica la gestión de bases de datos para los desarrolladores, sino que también crea un entorno altamente escalable para aplicaciones y servicios multiplataforma.

Los documentos o colecciones de documentos de MongoDB son las unidades básicas de datos. Con formato JSON binario (Java Script Object Notation), estos documentos pueden almacenar varios tipos de datos y distribuirse en múltiples sistemas.

Conceptos básicos

- Base de datos: gimnasio
- Colecciones: socios, clases, instructores, inscripciones
- Documentos: cada socio, clase o inscripción es un documento JSON3

Ejemplo de documentos

- Socio:

{ nombre: 'María López', edad: 28, membresia: 'Premium' }

- Clase:

{ nombre: 'Yoga', horario: 'Lunes 18:00', cupos: 20 }

- Instructor:

{ nombre: 'Pedro González', especialidad: 'Yoga y Pilates' }

Operaciones

- Insertar: insertOne(), insertMany()

```
db.socios.insertOne({  
    nombre: "María López",  
    edad: 28,  
    membresia: "Premium"  
})
```

```
db.socios.insertMany([  
    { nombre: "Juan Pérez", edad: 35, membresia:  
      "Básica" },  
    { nombre: "Lucía Fernández", edad: 22 },  
    { nombre: "Carlos Gómez", membresia: "VIP" }  
])
```

Operaciones

- Leer: find(), findOne()

```
// Todos los socios
```

```
db.socios.find()
```

```
// Socios Premium
```

```
db.socios.find({ membresia: "Premium" })
```

```
// Socios mayores de 30
```

```
db.socios.find({ edad: { $gt: 30 } })
```

```
//Devuelve uno
```

```
db.socios.findOne({ nombre: "Juan Pérez" })
```

Operaciones

- Actualizar: `updateOne()`, `updateMany()`

```
// Cambiar membresía de Juan Pérez a Premium
```

```
db.socios.updateOne(  
  { nombre: "Juan Pérez" },  
  { $set: { membresia: "Premium" } }  
)
```

```
// Subir todos los socios con membresía Básica a Standard
```

```
db.socios.updateMany(  
  { membresia: "Básica" },  
  { $set: { membresia: "Standard" } }  
)
```

Operaciones

- Eliminar: deleteOne(), deleteMany()

```
// Eliminar un socio específico
```

```
db.socios.deleteOne({ nombre: "Carlos Gómez" })
```

```
// Eliminar todos los socios con membresía  
Standard
```

```
db.socios.deleteMany({ membresia: "Standard" })
```

Operaciones

En MongoDB, cuando usás los métodos con **One** (findOne(), updateOne(), deleteOne()), devuelve solo el primer documento que cumple la condición.

Si hay varios, ignora los demás. El “primero” depende del orden natural en el que MongoDB tiene guardados los documentos (a menos que uses un .sort()).

Cuando usás un update con \$set, si el campo **no existe**, se crea automáticamente.

Consultas avanzadas y agregaciones

- \$lookup → unir colecciones

```
db.inscripciones.aggregate([
  {
    $lookup: {
      from: "socios",           // colección a unir
      localField: "socio_id",   // campo en inscripciones
      foreignField: "_id",       // campo en socios
      as: "datos_socio"         // nombre del nuevo array
    }
  }
])
```

Consultas avanzadas y agregaciones

- \$group → agrupar y contar

```
db.socios.aggregate([
  {
    $group: {
      _id: "$membresia", //campo por el que agrupo
      totalSocios: { $sum: 1 } //sumo de a uno
    }
  }
])
```

```
db.socios.aggregate([
  {
    $group: {
      _id: "$membresia", // agrupamos por membresía
      totalSocios: { $sum: 1 }, // contamos cantidad de socios
      totalCuotas: { $sum: "$cuota" }, // sumamos el valor de cuota
      promedioEdad: { $avg: "$edad" } // calculamos promedio de edades
    }
  }
])
```

Consultas avanzadas y agregaciones

- `$project` → seleccionar y transformar campos

```
db.socios.aggregate([
  {
    $project: {
      _id: 0,
      nombre: 1,
      edad: 1
    }
  }
])
```

- Resultado

```
{ "nombre": "María López", "edad": 28 }
{ "nombre": "Juan Pérez", "edad": 35 }
```

Consultas avanzadas y agregaciones

- Ejemplo: contar socios por membresía

```
db.socios.aggregate([
  {
    $project: {
      nombre: 1,
      edad: 1,
      mayorEdad: { $gte: ["$edad", 18] }
    }
  }
])
```

Índices

Son estructuras de datos que MongoDB crea aparte para localizar documentos más rápido.

Funcionan parecido al índice de un libro: en vez de leer todas las páginas (colección completa), va directo a la referencia.

Sin índice → la consulta hace un collection scan (revisa todo).

Con índice → busca solo en el índice, mucho más rápido

- `db.socios.createIndex({ nombre: 1 })`
- `db.clases.createIndex({ instructor_id: -1
 })`

Enlaces

Instalacion:

<https://www.mongodb.com/docs/manual/installation/>

Compilador Online:

<https://onecompiler.com/mongodb>

Documentacion:

<https://www.mongodb.com/docs/manual/>

Docker

- Instalar docker
<https://www.docker.com/products/docker-desktop/>
- Descargar la imagen de Mongo, desde el cmd ejecutar: `docker pull mongo`
- Crear el contenedor, desde el cmd ejecutar: `docker run -d --name mi-mongo -p 27017:27017 -v mongo-data:/data/db mongo`
- Para ver el estado docker `ps`
- Para conectarse a mongo `docker exec -it mi-mongo mongosh`